

EXAME ÉPOCA NORMAL (2ª PARTE, COM CONSULTA, DURAÇÃO: 75 min., 9 valores)

Nota: Nas perguntas seguintes, quando solicitado, deve usar troços de código Java essencial, podendo intermediar com linhas de comentário. Escrever código irrelevante com definições de *import* de *packages*, *static main*, *System.out.println*; *public* etc. é perda de tempo que lhe pode faltar para aspetos mais importantes.

9. [4 val.] Considere um cenário de uma agência de notícias que utiliza o rabbitMQ num IP de nome “agency” no porto 5672, num sistema de informação interno para publicação e consumo de notícias (strings de texto) sobre N temas (desporto, cultura, ...).

Para cada tema existe um *Exchange* do tipo FANOUT para onde são publicadas notícias e cada jornalista interno da agência usa uma aplicação (JApp) para receber as notícias dos temas que o jornalista trata.

A agência decidiu possibilitar a jornalistas externos (*extJApp*) também poderem consumir notícias, mas por questões de segurança não podem expor o IP do servidor RabbitMQ para o exterior. Assim decidiram encomendar o desenvolvimento de um servidor gRPC (*Frontend*) e uma aplicação *extJApp* com os seguintes requisitos:

- O contrato Ct deverá ter as seguintes operações:

// operação para obter a lista de temas

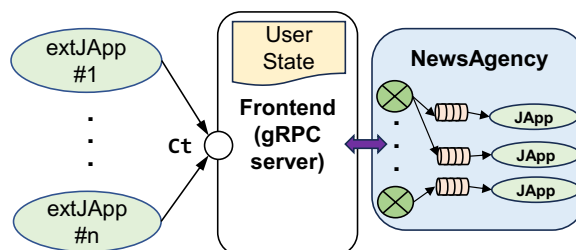
rpc getAllThemes(Void) returns (Themes)

// Operação que permite a um jornalista receber todas as notícias publicadas num tema

rpc registJournalistByTheme(RegistInfo) returns (stream News)

message Void { } **message Themes { repeated string theme=1; }**

message RegistInfo { string jid=1; string theme=2; } **message News { string someNews=1; }**



Assuma que os exchanges já estão criados e não tem de descrever ou implementar a aplicação JApp interna da agência

- a) [2 val.] Apresente o código Java essencial da classe que implementa o servidor **Frontend**, indicando claramente como implementar a manutenção de estado no servidor (*User State*) e a assinatura dos métodos que implementam o contrato, sem ser necessário indicar o código dos métodos nem o código do método *static main* de inicialização do servidor.
- b) [2 val.] Apresente o código Java essencial da operação **registJournalistByTheme**, de registo de jornalistas, indicando claramente como se processa a receção das notícias vindas do rabbitMQ e respetivo reenvio para a aplicação *extJApp* usada pelos múltiplos jornalistas externos.
10. [2.5 val.] [Não precisa apresentar código] Considere uma empresa que possui um computador cluster com 8 nós computacionais onde se pretende processar diariamente centenas de milhares de ficheiros PDF para recolher informação e armazená-la num servidor de Base de Dados existente numa máquina externa ao cluster. A aplicação que processa o PDF e armazena os dados na Base de Dados já existe disponível como um artefacto de nome *procPDF.jar* e recebe como argumento o nome de um PDF. Considerando os *middleware* gRPC, *Gluster* e *Docker*, proponha uma possível implementação de uma solução para o problema, usando um diagrama de componentes, a respetiva descrição dos contratos e as interações entre os componentes.
11. [2.5 val.] [Não precisa apresentar código] Considere um cenário onde se pretende implementar uma aplicação cliente/servidor para armazenar dados como pares (*Chave, Valor*). A componente cliente pode realizar duas operações. Uma operação em que dada uma *Chave* faz a leitura (*Read*) do *Valor* de um par e uma operação de escrita (*Write*) de um par (*K, V*). No caso de já existir um par com chave *K*, a operação de escrita faz *update* do valor anterior para o valor *V*. Por questões de balanceamento de carga e de tolerância a falhas devem existir múltiplos servidores que mantêm réplicas de todos os pares. Considerando os diferentes tipos de *middleware* e tecnologias estudadas, proponha uma possível arquitetura, usando um diagrama de componentes, a respetiva descrição dos contratos e as interações entre as componentes. Considere os seguintes requisitos: i) O cliente pode em qualquer momento obter a lista de localização dos múltiplos servidores; ii) Uma operação *Write* de um par é pedida a um único servidor, mas terá de ser efetivada o mais consistente possível em todos os servidores; iii) Em cada servidor, os dados devem ser armazenados de forma persistente.